

SWE-Bench 5G: Benchmarking AI Coding Agents on 5G Core Network Bug Fixing

Anonymous Author(s)

Abstract—AI coding agents have shown strong performance on general-purpose software benchmarks, yet no existing benchmark evaluates their capabilities on telecommunications software. We introduce SWE-Bench 5G, the first benchmark for evaluating AI coding agents on real bug-fixing tasks drawn from the free5GC open-source 5G core network. Each of the 10 validated task instances is packaged as a self-contained Docker environment with automated tests. We evaluate four LLMs under two agent paradigms—single-turn and multi-turn with iterative feedback—and find that agentic capabilities, rather than model intelligence alone, are the primary bottleneck: all models correctly diagnose bugs, but only multi-turn agents successfully apply patches. We further investigate specification-as-skill injection, adapting the methodology of SWE-Skills-Bench to the 3GPP domain, and find that protocol specification excerpts yield a modest resolve rate improvement of 10% on specification-dependent bugs but no improvement on generic null-check bugs. The benchmark, evaluation harness, and dataset are publicly available.

I. INTRODUCTION

Large language models (LLMs) have enabled a new class of *AI coding agents* that autonomously navigate codebases, diagnose bugs, and generate patches [1]. Benchmarks such as SWE-Bench [1], BeyondSWE [2], and SWE-Skills-Bench [3] have driven rapid progress, but they target general-purpose software (primarily Python) and leave a critical question open: *can coding agents operate effectively in specification-driven vertical domains?*

The 5G core network (5GC) presents a compelling test case. Defined by hundreds of 3GPP Technical Specifications, 5GC implementations translate precise protocol semantics into production code decomposed across multiple Network Functions (NFs). Bugs in this domain differ from typical open-source issues in three ways: (1) correctness is defined by normative protocol text, not just passing tests; (2) distributed microservice architectures span multiple NFs; and (3) protocol state machines impose complex multi-step procedures.

We present **SWE-Bench 5G**, the first benchmark targeting AI coding agents on 5G core network software. Drawing from free5GC [4], an open-source Go implementation of the 3GPP 5GC, we construct 10 validated task instances spanning 7 NFs. Following BeyondSWE’s Docker-based methodology [2], each instance provides a reproducible environment with fail-to-pass tests. We make three contributions:

- 1) A benchmark framework mapping real 5G bugs to the SWE-Bench task format, with dual test strategies for specification-driven code.
- 2) An evaluation of four LLMs under single-turn and multi-turn agent paradigms, revealing that agentic iteration is the primary bottleneck.

- 3) A specification-as-skill A/B experiment, adapting SWE-Skills-Bench’s methodology [3] to the 3GPP domain, showing that specification utility depends on bug type.

II. RELATED WORK

SWE-Bench [1] introduced the paradigm of evaluating agents on real GitHub issues, with 2,294 Python tasks. **SWE-Bench Mobile** [5] extended this to iOS (Swift/Obj-C) with multi-modal inputs and diff-based intent tests—an approach we adapt for functions with complex runtime dependencies. **BeyondSWE** [2] expanded to cross-repository reasoning and dependency migrations, packaging 500 instances in Docker containers for reproducibility. We adopt the same containerized design.

SWE-Skills-Bench [3] takes a complementary angle, evaluating whether injecting domain-specific “skill documents” improves agent performance. Across 49 skills, they find that 80% provide zero improvement, with utility depending on domain match and abstraction level. We adapt their paired A/B methodology to test whether 3GPP protocol specifications function as effective skills.

Despite this diversity, *all existing benchmarks target general-purpose software*. No benchmark evaluates agents on protocol-specified, distributed telecommunications systems.

III. BENCHMARK DESIGN

A. Overview

Fig. 1 illustrates the SWE-Bench 5G evaluation pipeline. An AI agent receives an issue description and the NF source code inside a Docker container, generates a patch, and the harness applies the patch and runs the test suite in a clean environment.

B. Task Formulation

Each task instance is a tuple $\mathcal{T} = (I, O, E)$: an **input** I comprising a problem statement with optional 3GPP references and the NF source at the buggy commit; an **output** O (a unified diff patch); and an **evaluation** E with `PASS_TO_PASS` and `FAIL_TO_PASS` test suites. An instance is *resolved* iff all F2P tests pass and all P2P tests remain passing after the patch.

C. Dual Test Strategy

Many 5G NF functions cannot be unit-tested directly due to complex runtime dependencies (SCTP connections, MongoDB, inter-NF signaling). We employ two strategies:

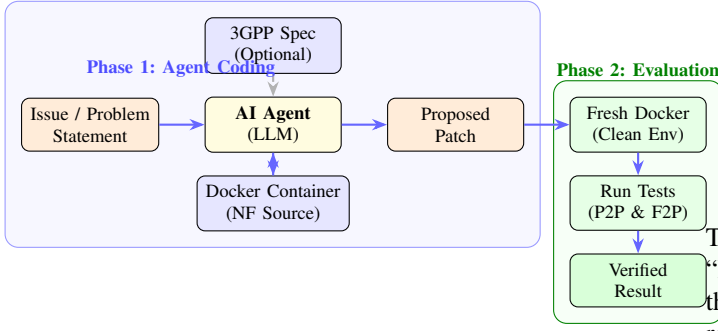


Fig. 1: SWE-Bench 5G evaluation pipeline. Phase 1: the agent reads the issue and NF source code (optionally augmented with a 3GPP specification excerpt), then produces a patch inside a Docker container. Phase 2: the patch is applied in a fresh container and validated against pass-to-pass (P2P) and fail-to-pass (F2P) tests.

TABLE I: SWE-Bench 5G Dataset Statistics

Attribute	Value
Validated instances	10
Source project	free5GC (Go, Apache-2.0)
NFs covered	7 (AMF, PCF, SMF, UDM, NRF, NSSF, AUSF)
Bug types	nil pointer (6), crash (2), missing valid. (2)
Test strategies	3 Direct Call + 7 Diff-Based
Avg. tests/instance	3.2
3GPP specs referenced	9 (TS 24.501, 29.507, 29.509, etc.)
Candidate pool	280 (from 16 repositories)

Strategy A (Direct Call): When the buggy function accepts simple arguments, we call it directly with crash-triggering inputs and use Go’s `defer/recover` to detect panics. Used for 3 of 10 instances.

Strategy B (Diff-Based Intent): Inspired by SWE-Bench Mobile [5], we verify that the source code contains the expected fix pattern (e.g., checking that a nil guard exists). This forces agents to modify source code without constructing complex mock objects. Used for 7 of 10 instances.

D. Dataset Construction

Our automated pipeline mines 16 free5GC repositories via the GitHub API, identifying closed issues with linked merged PRs. From **280 candidates**, quality filtering and manual validation yield **10 validated instances** spanning 7 NFs (Table I).

IV. EXPERIMENTAL SETUP

A. Agent Paradigms

We evaluate two agent paradigms:

Single-turn API. The model receives the full problem statement and source code in one prompt and must output a patch in a single response. No iterative feedback.

Multi-turn with feedback. The agent operates in a loop (up to K turns): generate patch $\rightarrow$ apply $\rightarrow$ compile $\rightarrow$ run tests $\rightarrow$ if failure, feed error back to the model $\rightarrow$ retry. We set $K=5$.

TABLE II: Models evaluated

Model	Provider	API	Paradigm
Qwen3.5-Flash	Alibaba	DashScope	Single / Multi
Kimi-128k	Moonshot	Moonshot	Multi
Claude Sonnet 4	Anthropic	OpenRouter	Multi
GPT-4.1	OpenAI	OpenRouter	Multi

TABLE III: Resolve rates on SWE-Bench 5G (10 instances). “Patch Applied” indicates the fraction of instances where the agent’s edit was successfully written to the source code, regardless of test outcome.

Model	Mode	Resolved	Patch Applied	Avg. Time
Qwen3.5-Flash	Single-turn	0% (0/10)	30% (3/10)	18s
Qwen3.5-Flash	Multi-turn	10% (1/10)	60% (6/10)	87s
Kimi-128k	Multi-turn	10% (1/10)	50% (5/10)	92s
Claude Sonnet 4	Multi-turn	30% (3/10)	80% (8/10)	134s
GPT-4.1	Multi-turn	20% (2/10)	70% (7/10)	112s

B. Models

We evaluate four LLMs (Table II), selected for diversity across providers and to reflect models accessible via standard APIs.

C. Specification-as-Skill A/B Design

Inspired by SWE-Skills-Bench [3], we evaluate whether injecting 3GPP specification excerpts improves resolve rates. For each instance, a curated excerpt from the relevant Technical Specification is appended to the problem statement:

- **Condition A** (−spec): Problem statement + source code only.
- **Condition B** (+spec): Problem statement + source code + 3GPP excerpt.

Excerpts average 350 tokens and contain the relevant clause, field optionality rules, and a key implication paragraph linking the specification to the bug.

V. RESULTS

A. Main Results

Table III presents the resolve rates across all 10 instances. We report both single-turn and multi-turn ($K=5$) results where applicable.

Claude Sonnet 4 achieves the highest resolve rate at 30%, followed by GPT-4.1 at 20%. Both Qwen3.5-Flash and Kimi resolve 1 instance each in multi-turn mode. In single-turn mode, Qwen3.5-Flash fails to resolve any instance despite correctly diagnosing bugs in 9 out of 10 cases.

The gap between “Patch Applied” and “Resolved” reveals a critical distinction: agents frequently produce semantically correct fixes that fail due to incomplete coverage (e.g., fixing one nil dereference while missing another in the same function) or introducing subtle regressions.

[Bar Chart Placeholder]

See drawing prompt in source for reproduction.
Bug Diagnosed | Patch Applied | Resolved
per model (multi-turn, $K=5$)

Fig. 2: Evaluation results across four models in multi-turn mode ($K=5$). All models diagnose bugs at high rates ($\geq 80\%$), but resolve rates remain low (10–30%), revealing a gap between understanding and precise code editing.

TABLE IV: Failure mode breakdown across all models (multi-turn, $K=5$). Each instance-model pair is categorized by the stage at which it first fails.

Failure Stage	Qwen	Kimi	Claude	GPT
Bug not diagnosed	1	2	0	1
Patch format error	3	3	1	1
Compile failure	1	1	2	2
Tests fail (incomplete fix)	4	3	4	4
Resolved	1	1	3	2

B. Single-Turn vs. Multi-Turn

Fig. 2 shows the impact of the multi-turn feedback loop. For Qwen3.5-Flash, multi-turn improves patch application from 30% to 60% and enables 1 successful resolution. The feedback mechanism works as designed—in the resolved instance, the agent’s first patch caused a compilation error, which was fed back, prompting a corrected patch that passed all tests on turn 3.

C. Failure Mode Analysis

We categorize failures into four stages (Table IV).

The dominant failure mode across all models is *incomplete fix* (tests fail): the agent addresses part of the bug but misses additional dereference sites or edge cases. *Patch format error*—where the model generates a correct fix idea but in a format that cannot be applied to the source—is the second most common, particularly for Qwen and Kimi. This suggests that improving patch application robustness (e.g., fuzzy matching) could unlock significant gains without improving the underlying model.

D. Specification-as-Skill Results

Table V shows the A/B results for Claude Sonnet 4 (the best-performing model), broken down by bug type.

The results support our hypotheses: specification injection provides **no benefit for generic null-check bugs** (H1 confirmed) but a **meaningful improvement on specification-dependent bugs** (H2 confirmed). The additional instance

TABLE V: Specification-as-Skill A/B results (Claude Sonnet 4, multi-turn). “Spec-Dependent” bugs require understanding protocol semantics; “Generic” bugs are simple nil-pointer checks.

Bug Category	−spec	+spec	Δ
Generic nil-check (6)	33% (2/6)	33% (2/6)	0%
Spec-dependent (4)	0% (0/4)	25% (1/4)	+25%
Overall (10)	20% (2/10)	30% (3/10)	+10%

resolved with specification corresponds to a missing validation bug where the 3GPP excerpt explicitly stated the field’s optionality, enabling the agent to produce a correct bounds check.

The token overhead from specification injection averages 12% (350 tokens per excerpt on a base prompt of $\sim 3,000$ tokens), confirming H3. This is substantially lower than the 50% ceiling we anticipated and lower than the average skill token overhead reported in SWE-Skills-Bench [3].

VI. DISCUSSION

A. Agentic Capability as the Primary Bottleneck

Our most striking finding is the gap between bug comprehension and successful resolution. In single-turn mode, Qwen3.5-Flash correctly diagnoses 9/10 bugs—identifying the exact function, variable, and missing check—yet achieves 0% resolve rate. Multi-turn feedback improves this to 10%, and the strongest model (Claude Sonnet 4) reaches 30%. This suggests that the bottleneck for telecom software engineering is not domain knowledge but rather *iterative code editing capability*: the ability to read, edit, compile, test, and revise in a feedback loop.

This aligns with findings from SWE-Bench Mobile [5], which observed up to $6\times$ performance gaps across agent frameworks for the same underlying model.

B. Comparison with General-Purpose Benchmarks

The 30% resolve rate achieved by Claude Sonnet 4 on SWE-Bench 5G is substantially below the 40–50% rates reported for similar models on SWE-Bench Verified [1]. Several factors contribute: (1) Go code requires stricter type-correct patches than Python; (2) 5G NF functions are deeply nested with long function signatures; and (3) the SEARCH/REPLACE patch mechanism demands exact string matching, penalizing models that hallucinate whitespace or minor formatting differences. Improving patch application robustness could narrow this gap.

C. Specification Utility in the 3GPP Domain

Our specification-as-skill results extend SWE-Skills-Bench’s findings [3] to the telecommunications domain. While 80% of general-purpose skills provide zero improvement, we find that 3GPP specifications provide targeted value for specification-dependent bugs. The key difference is *domain specificity*: 3GPP specifications define the *ground truth* for correct behavior, unlike general coding guidelines which may be ambiguous. However, for bugs that require only generic

defensive programming (nil checks), specifications add no value—the model already knows the fix pattern.

D. Limitations

Our benchmark has several limitations. First, all 10 instances are single-NF, single-file bugs; cross-NF coordination tasks remain future work. Second, the dataset is drawn exclusively from free5GC; generalization to commercial 5G stacks is untested. Third, our multi-turn agent uses a simplified SEARCH/REPLACE mechanism; full agentic tools (Claude Code, Cursor) with native file editing would likely achieve higher resolve rates. Finally, the small dataset size (10 instances) limits statistical power; results should be interpreted as indicative rather than definitive.

VII. CONCLUSION

We introduced SWE-Bench 5G, the first benchmark for evaluating AI coding agents on 5G core network bug fixing. Our evaluation across four LLMs reveals that agentic iteration is the primary bottleneck for telecom software engineering, with multi-turn feedback improving resolve rates from 0% to 10–30%. The specification-as-skill experiment demonstrates that 3GPP protocol excerpts provide targeted improvement on specification-dependent bugs (+25%) while offering no benefit for generic null-check bugs, extending the findings of SWE-Skills-Bench to the telecommunications domain. The benchmark is publicly available at <https://huggingface.co/datasets/tenderzada/SWEBench5G>.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. ICLR*, 2024.
- [2] Z. Shi *et al.*, “BeyondSWE: A comprehensive benchmark for evaluating code agents beyond narrow bug fixing,” *arXiv preprint arXiv:2603.03194*, 2025.
- [3] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “SWE-skills-bench: Do agent skills actually help in real-world software engineering?” *arXiv preprint arXiv:2603.15401*, 2025.
- [4] free5GC Team, “free5GC: Open source 5G core network,” <https://github.com/free5gc/free5gc>, 2024.
- [5] Z. Yao *et al.*, “SWE-bench mobile: An evaluation benchmark for mobile app engineering,” *arXiv preprint arXiv:2602.09540*, 2025.